

Orbis: An Intelligent Academic Support System with AI and RAG

by

Atakan Gül, 121200152

Arda Kaan Yıldız, 122200045

Supervised by

Özgür Özdemir

Submitted to the

Faculty of Engineering and Natural Sciences

in partial fulfillment of the requirements for the

Bachelor of Science

in the

Department of Computer Engineering

June, 2026

Abstract

University academic systems often function merely as static record-keeping databases, lacking proactive support for students navigating complex degree requirements. This project, 'Orbis', introduces an intelligent academic support system designed to bridge this gap using Artificial Intelligence (AI) and Retrieval-Augmented Generation (RAG). The core of Orbis is an Orchestrator-led event management pipeline that autonomously monitors a student's dynamic academic state against a complex vector database of university regulations, triggering timely notifications for critical milestones. To achieve high fidelity, we engineered a taxonomy generation pipeline utilizing Shannon entropy tail-signal classification and MiniBatchKMeans clustering, organizing 60,649 document chunks into 125 high-quality leaf nodes. Our pipeline features a two-pass adversarial LLM review, evaluating 323 candidate rules and extracting 178 highly actionable obligations with a 55% acceptance rate, and matches them against 4 distinct student profiles to generate 176 personalized assignments. We evaluate each subsystem against a gold ground-truth set manually validated by the authors. The event-extraction stage achieves precision 0.978 / recall 0.574 / F_1 0.723 over 323 candidates; the assignment-matching stage achieves F_1 0.51 over 712 (profile, obligation) pairs. An agentic submission-review subsystem evaluated on 32 deterministically generated cases attains 93.8% accuracy with perfect recall and full (4/4) resistance to in-document prompt injection. The remaining roadmap includes direct integration with the university's Student Information System (SIS) and injection-hardened submission review.

Contents

Abstract	1
Contents	2
List of Figures	4
List of Tables	5
List of Abbreviations	7
1 Introduction	1
2 Related Works	1
3 Dataset	4
4 Theoretical Background	5
4.1 Shannon Entropy for URL Classification	5
4.2 Vector Embeddings and Cosine Similarity	6
4.3 MiniBatchKMeans and Ward HAC	6
5 Proposed Methodology	6
5.1 Data Categorization & Taxonomy Strategy	6
5.2 The Orchestrator-Led Event Pipeline	7
5.3 Contextual vs. SQL Assignment Matching	8
5.4 Agentic Assignment Submission Evaluation	9
6 Experimental Setup	11
6.1 Framework and Deployment	11
6.2 Performance Metrics	11
7 Experiments & Discussion	12
7.1 Extraction Throughput & Quality Funnel	12
8 Results	12

9	Quantitative Evaluation	13
9.1	Gold Ground-Truth Construction	14
9.2	Event Extraction Results	15
9.3	Assignment Matching Results	16
9.4	Submission Agent Results	16
9.5	Limitations	18
10	Realistic Constraints	18
10.1	Social, Environmental and Economic Impact	18
10.2	Cost Analysis	18
10.3	Standards	20
10.3.1	NSPE Code of Ethics	20
10.3.2	IEEE Standards	20
10.4	Risk Analysis	20
11	Conclusion & Future Work	21
12	Contributions	22
	References	23

List of Figures

1	Data Categorization and Taxonomy Pipeline Architecture	7
2	Sequential flow and architecture of the Orbis Proactive Event Pipeline. The orchestrator owns all run state; workers are stateless and communicate only via the orchestrator. The reviewer is invoked only for ambiguous candidates.	8
3	Agentic submission-evaluation flow. A deterministic file gate rejects empty, corrupt, or unreadable files without invoking the LLM. The agent then reasons in two LLM calls: it first <i>decomposes</i> the assignment into a requirement checklist, then <i>judges the document against each requirement individually</i> with cited evidence before synthesizing a verdict. A deterministic keyword <code>line_review</code> is retained only as a fallback when decomposition fails. Every stage is streamed over Server-Sent Events so the UI can render the source text, the requirements, and each per-requirement finding live. Every rejection (early-stage or LLM-based) is recoverable through the <code>flag_for_review</code> appeal path, which routes the case to a human reviewer instead of terminating the student’s submission attempt.	10

List of Tables

1	Positioning of Orbis against related categories of academic-support systems. <i>Push</i> means the system surfaces an obligation before a student asks. <i>Unstructured rules</i> means handling free-text regulation, not only schema-based requirements.	3
2	Official baseline and comparator validity. <i>Direct</i> means the same task and data family; <i>partial</i> means a related subsystem metric but different domain or output schema; <i>contextual</i> means useful for positioning only.	4
3	Dataset and post-processing statistics. The Proactive Event System operates only on the 935 regulatory chunks; all other categories are routed to the RAG search index.	5
4	Event Extraction Pipeline Funnel	12
5	Assignments generated per real student profile, broken down by urgency and matching path. “SQL” counts deterministic conditional matches; “CTX” counts contextual LLM matches.	13
6	Metric provenance and limitations. Artifact paths are reported relative to the repository.	15
7	Confusion matrix of the event extraction pipeline against gold ground truth ($N=323$).	15
8	Binary classification metrics for the event extraction stage. Positive class = true obligation accepted by the system.	15
9	Per-profile assignment matching scores against gold ground truth ($N=712$ pairs: 4 profiles $\times$ 178 accepted obligations).	16
10	Submission agent results stratified by case type. Seven of the eight strata are perfectly handled; only partial-draft detection remains imperfect. . . .	17
11	Submission agent confusion matrix on $N=32$ cases (positive class: approve). Zero false negatives: no genuine submission was rejected. Both false positives are <code>flag_partial</code> drafts.	17
12	Submission agent binary classification metrics (positive class: approve), measured on the 32-case ground-truth set against the current two-call agentic reviewer.	17
13	Submission Agent Ablation Study: Redesigning the reviewer significantly lifted in-document prompt-injection resistance because the agent now explicitly verifies whether requirements are met, bypassing embedded adversarial instructions.	18

14	Per-stage model choice and OpenRouter pricing. Cheaper models are used for high-volume bulk labeling; a stronger model is reserved for adversarial review and submission reasoning.	19
----	---	----

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

CI/CD Continuous Integration/Continuous Deployment

DB Database

HAC Hierarchical Agglomerative Clustering

LLM Large Language Model

RAG Retrieval-Augmented Generation

TF-IDF Term Frequency-Inverse Document Frequency

1 Introduction

In universities, students face a significant cognitive load managing complex degree rules, course selections, and deadlines. Traditional academic platforms are often designed as static databases. They store information efficiently but fail to interpret it for the student’s benefit. Students generally do not miss deadlines because they cannot search a system; they miss them because they do not know what they are supposed to be looking for. This creates a "synchronization gap" between student needs and institutional data.

Our project, 'Orbis', offers a novel solution by shifting the academic paradigm from static storage to autonomous orchestration. Orbis is built to act as a proactive academic agent. While standard AI integrations in education wait for user prompts, the core innovation of Orbis is its autonomous evaluation of a student’s specific state against unstructured, complex university regulations.

Crucially, the architecture of Orbis is designed for scalability. The system relies on a strictly orchestrated workflow to execute its primary use case: the **Proactive Event System**. This autonomous pipeline extracts strict, assignable obligations from regulatory documents and creates normalized events, allowing the system to trigger timely, unprompted notifications for students regarding their specific academic requirements.

The rest of the work is organized as follows. Section 2 overviews related works. Section 3 details the dataset. Section 4 provides the theoretical background. Section 5 presents the proposed methodology. Section 6 describes the experimental setup. Sections 7 and 8 discuss experiments and results. Section 9 provides quantitative evaluation. Section 10 outlines realistic constraints. Section 11 concludes the paper, and Section 12 details the authors’ contributions.

2 Related Works

Academic support systems have evolved from static record-keeping into AI-mediated tools, but most existing systems remain reactive: they answer student questions but do not autonomously enforce regulations against a student’s state. We position Orbis against six lines of work.

RAG-based academic chatbots. Retrieval-Augmented Generation combines a generative LLM with a non-parametric retrieval store, grounding answers in source documents and reducing hallucination [1]. A systematic review of advising chatbots screened 128 studies and retained 11 advising-chatbot papers, finding that most systems target admission or academic-advising dialogue rather than autonomous obligation enforcement [2]. More recent advising-LLM work reports QA-style metrics such as BLEU, ROUGE-L, and

student satisfaction ratings [3]. These systems serve well as on-demand search interfaces, but operate strictly in a pull mode — a student must already know that a question is worth asking. Orbis instead operates in a push mode: it parses regulations into structured obligations and matches them against profiles before the student asks.

Degree-audit systems. Tools such as DegreeWorks and Stellic encode degree requirements as deterministic rules and audit a transcript against them. The Degree Audit Reporting System (DARS), for example, has been studied as an advising and registration-support system rather than as an information-extraction benchmark [4]. Such systems are excellent for objective conditions (credit counts, GPA thresholds) but cannot reason about unstructured policy text (procedural requirements, deadline-bearing obligations, exceptions written in prose). Orbis complements deterministic audit with a contextual LLM path that operates over free-text regulation.

Regulatory information extraction. Prior regulatory IE work is closer to Orbis’s event-extraction subsystem, but not to the full Orbis pipeline. Zhang and El-Gohary report 0.969 precision and 0.944 recall when extracting quantitative requirements from the 2009 International Building Code using semantic NLP rules [5]. Recent legal-obligation extraction work on the EU AI Act reports 93% precision for obligation filtering and over 99% accuracy for classifying obligation type, addressee, and predicate [6]. These results are useful precision/recall comparators for the extraction stage, but their domains, source documents, and output schemas differ from university-student obligation assignment.

Local context: HermiOne. Istanbul Bilgi University has deployed “HermiOne,” a support chatbot built by the university’s IT department. Based on publicly observable behavior at the time of writing, HermiOne appears to operate primarily as a reactive question-answering chatbot rather than as a proactive regulation-enforcement agent; a direct comparison would require access to its design documentation, which we did not have. Orbis is intentionally complementary: HermiOne handles the conversational interface, while Orbis produces the underlying structured event stream that a future iteration of HermiOne could subscribe to.

Automated submission assessment. LLM-assisted grading systems provide a useful comparison point for the submission-review subsystem, but their metrics usually measure grade prediction rather than safety-oriented accept/reject screening. JorGPT, for instance, evaluates programming-assignment grading over 672 submissions and reports adjusted $R^2=0.9156$, mean absolute error 0.4579, and a reduction from over 300 hours of manual work to under 15 minutes of automated processing [7]. Orbis’s submission agent is narrower: it does not assign grades, but checks whether a submitted artifact satisfies assignment requirements and preserves an appeal path.

LLM-as-a-judge for evaluation. Because labeled gold sets for university-specific regulation extraction do not exist publicly, we adopt the LLM-as-a-judge methodology [8], in which an independent LLM is used as a scalable proxy for human labeling and validated against a second model from a different family. This methodology underlies our evaluation in Section 9.

Table 1 summarizes the comparison.

System type	Pull/push	Unstructured rules	Student-specific	Auto-notify
University FAQ chatbot	Pull	No	No	No
RAG academic chatbot	Pull	Yes	No	No
Deterministic degree audit	Push	No	Yes	Partial
HermiOne (reactive)	Pull	Yes	Partial	No
Orbis (this work)	Push	Yes	Yes	Yes

Table 1: Positioning of Orbis against related categories of academic-support systems. *Push* means the system surfaces an obligation before a student asks. *Unstructured rules* means handling free-text regulation, not only schema-based requirements.

No exact end-to-end baseline was found for the combined task Orbis addresses: proactive extraction of university regulations, profile-specific obligation assignment, and submission review with appeal. Therefore, Table 2 compares Orbis against official prior work at the subsystem level and explicitly marks whether each comparison is direct, partial, or contextual. This distinction is important: we do not claim that Orbis “beats” systems evaluated on different tasks; we use them to position the contribution and to choose defensible metrics.

Official prior work	Related Orbis subsystem	Reported metric or scope	Comparison type	How it is used in this report
Assayed et al. systematic review [2]	Advising-chatbot landscape	128 articles screened; 11 included advising-chatbot studies	Contextual	Shows that advising chatbots are active research systems, but mostly dialogue/admission/advising tools rather than proactive rule-assignment pipelines.
Ismail academic-advising LLM [3]	Advising chatbot	BLEU 0.45→0.78; ROUGE-L 0.85; 50-student ratings 4.6–4.8/5	Contextual	Provides QA-chatbot metrics; not directly compared because Orbis is evaluated on extraction, matching, and submission screening rather than answer generation.
Wick and Wallingford DARS study [4]	Degree audit	Qualitative analysis of DARS for advising and registration support	Contextual	Positions deterministic degree-audit systems as complements, not direct NLP baselines.
Zhang and El-Gohary regulatory IE [5]	Event extraction	Precision 0.969; recall 0.944 on building-code quantitative requirements	Partial	Provides a precision/recall comparator for regulatory extraction, with different domain and rule schema.
Galli et al. AI Act obligation extraction [6]	Event extraction	93% obligation-filtering precision; >99% classification accuracy for extracted obligation fields	Partial	Provides a modern legal-obligation extraction comparator; not compared to assignment matching or submission review.
JorGPT LLM grading [7]	Submission assessment	Adjusted R^2 0.9156; MAE 0.4579; 672 submissions; 300+ hours → under 15 minutes	Partial	Provides grading/workload context; Orbis reports binary accept/reject safety metrics instead of grade-regression metrics.
Prior Orbis single-call reviewer	Submission review	Accuracy 0.781; injection resistance 1/4	Direct	Internal ablation baseline for the current two-call reviewer reported in Section 9.4.

Table 2: Official baseline and comparator validity. *Direct* means the same task and data family; *partial* means a related subsystem metric but different domain or output schema; *contextual* means useful for positioning only.

3 Dataset

The foundation of Orbis is its institutional `knowledge_base`, constructed from 80 official source URLs under the `bilgi.edu.tr` domain. The dataset and its post-processing artifacts are summarized in Table 3.

Quantity	Value
Source URLs crawled	80
Raw document chunks	60,649
Languages observed	English, Turkish
Top-level taxonomy categories (LLM-mapped)	11
Leaf-level taxonomy nodes	125
Regulatory leaves (post-overlay)	29
Regulatory chunks (event-pipeline input)	935
Accepted obligations (<code>regulation_rules</code>)	178
Student profiles (<code>user_profiles</code>)	4

Table 3: Dataset and post-processing statistics. The Proactive Event System operates only on the 935 regulatory chunks; all other categories are routed to the RAG search index.

The crawl is intentionally heterogeneous: it contains formal regulations (student handbooks, dual-major directives, internship procedures), administrative announcements, event pages, and upload directories. The corpus is bilingual, which complicates deduplication — the same regulation often exists under both `/en/academics/...` and `/tr/akademik/...` paths. Chunking is performed at ingest time using a semantic splitter with a soft 800-token target and a 100-token overlap; each chunk is embedded once with a local MiniLM model and stored in `knowledge_base_embeddings` alongside the source URL, content hash, and categorical metadata. The regulation subset is isolated via the four-step taxonomy pipeline of Section 5.1, with a final regex overlay that promotes any chunk whose source URL matches a known regulation path into the `regulation` or `regulation_document` category, preventing event-pipeline cross-contamination from non-binding content. Instead, evaluation is performed against a constructed Gold Ground-Truth testing set, detailed in Section 9.

4 Theoretical Background

To successfully isolate and evaluate this dataset, Orbis relies on several foundational machine learning and information theory algorithms.

4.1 Shannon Entropy for URL Classification

To classify opaque URL structures, we measure the diversity of URL "tails" using Shannon Entropy [9]. A high entropy value indicates that a URL directory is semantically

informative, whereas low entropy implies repetitive or dynamic templates.

$$H = - \sum_{i=1}^n p_i \cdot \log_2(p_i)$$

Where p_i is the probability of a specific URL pattern occurring within a given parent group.

4.2 Vector Embeddings and Cosine Similarity

Text chunks are transformed into high-dimensional numerical vectors. To measure the semantic similarity between two chunks (or a user query and a document), we calculate the Cosine Similarity.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Prior to clustering, we perform L2-Normalization on all vectors ($\hat{v} = \frac{v}{\|v\|}$), converting cosine distance into Euclidean distance on a unit sphere, optimizing it for K-Means processing.

4.3 MiniBatchKMeans and Ward HAC

For semantic discovery of opaque URLs, we deploy MiniBatchKMeans, which utilizes random mini-batches to achieve linear time complexity $\mathcal{O}(n)$ suitable for large datasets [10]. This is followed by Ward Hierarchical Agglomerative Clustering (HAC) on the centroids. Ward’s method merges clusters to minimize the total within-cluster variance [11].

5 Proposed Methodology

The Orbis architecture is divided into two major phases: Data Categorization (Taxonomy) and the Orchestrator-led Event Pipeline.

5.1 Data Categorization & Taxonomy Strategy

Standard RAG systems suffer degraded accuracy when mixing formal rulebooks with informal forum posts. Orbis mitigates this via a four-step semantic discovery pipeline.

1. **Step 1: Rule Engine:** Evaluates URL prefixes calculating semantic and dynamic ratios. Groups with high dynamic ratios (e.g., UUIDs) are rejected.

2. **Step 2: Tail Signal Classification:** Computes the Shannon entropy of URL tails. High entropy groups are classified as KNOWN.
3. **Step 3: Semantic Discovery:** The 21,324 UNKNOWN documents undergo L2-normalization and MiniBatchKMeans clustering. Ward HAC generates 52 fine-level labeled clusters based on TF-IDF scoring.
4. **Step 4: AI Taxonomy Mapping:** An LLM maps the deduplicated groups into a closed vocabulary of 11 top-level categories and 96 specific leaf nodes. A targeted regex overlay isolates the 935 regulation chunks into 29 distinct regulatory leaves.

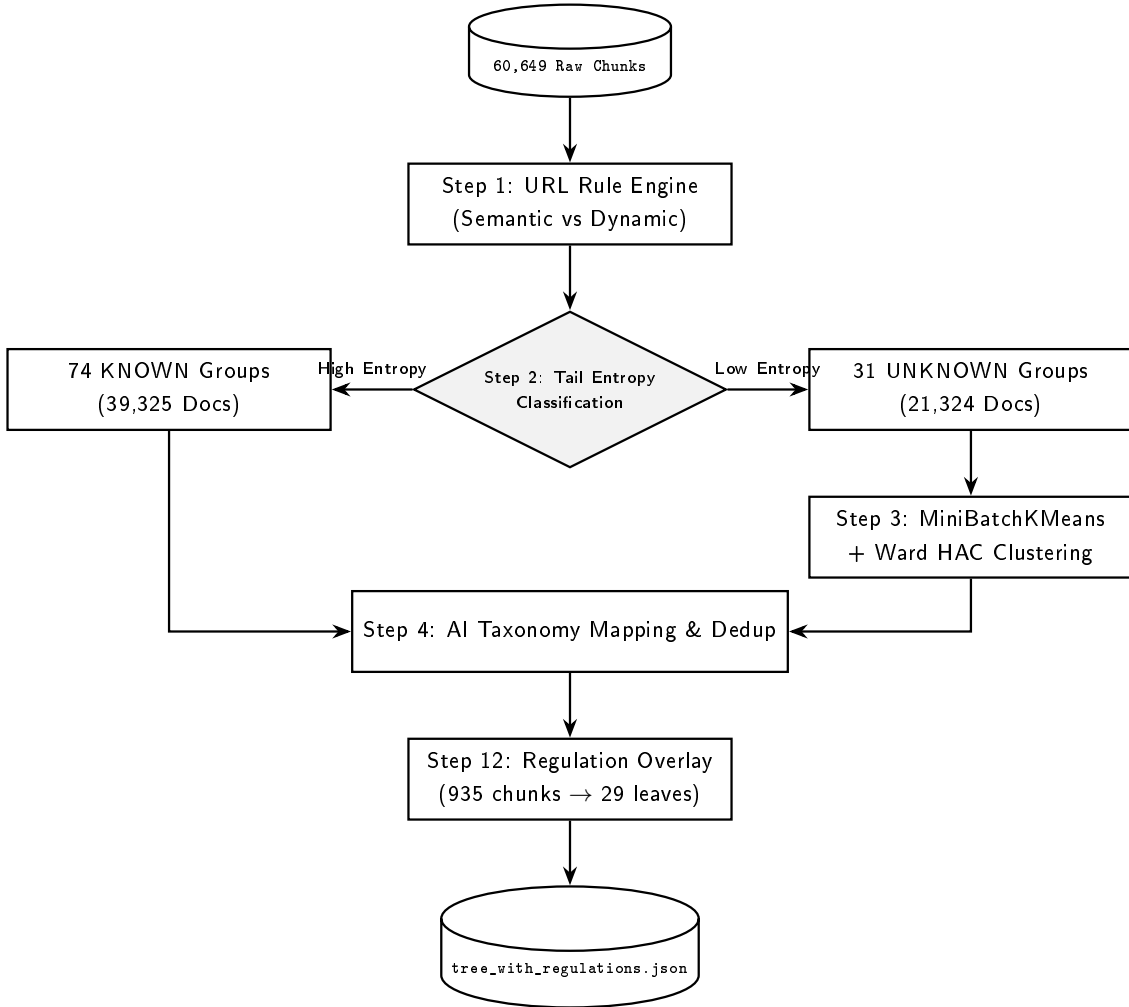


Figure 1: Data Categorization and Taxonomy Pipeline Architecture

5.2 The Orchestrator-Led Event Pipeline

Unlike decentralized multi-agent systems, Orbis utilizes a centralized `EventPipelineOrchestrator`. This stateful hub controls stateless worker agents (`SearchAgent`, `ReasoningAgent`, `EventCreator`) to ensure database consistency and pipeline idempotency.

Figure 2 summarizes the proactive event pipeline. The system, led by a central orchestrator, leverages specialized agents to locate sources, extract candidates, validate them, and finally store the parsed, unambiguous rules into the database while maintaining rigorous execution states.

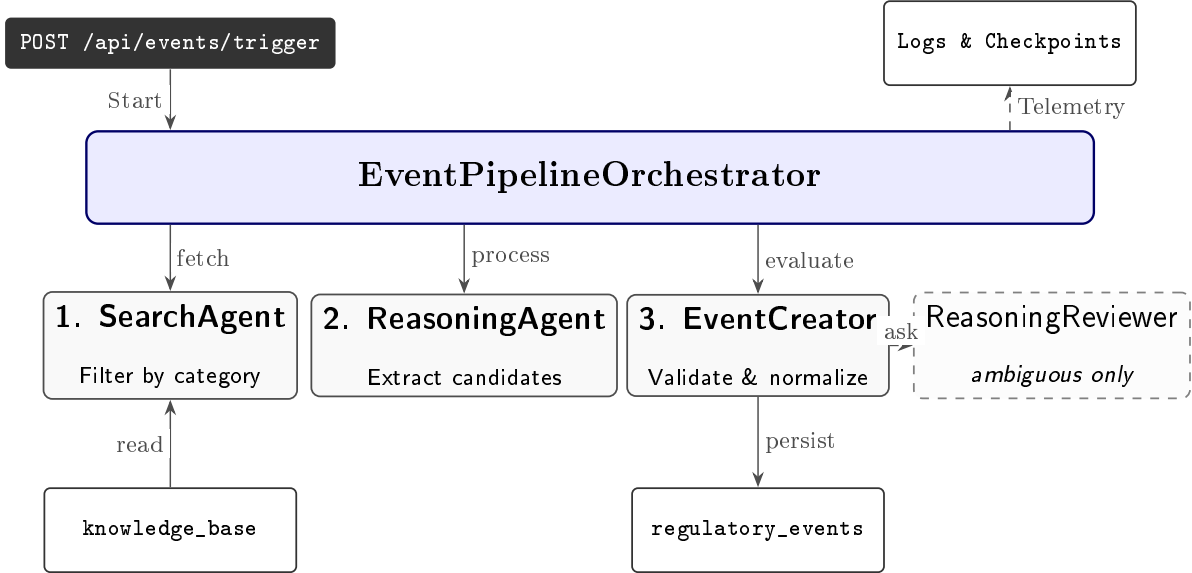


Figure 2: Sequential flow and architecture of the Orbis Proactive Event Pipeline. The orchestrator owns all run state; workers are stateless and communicate only via the orchestrator. The reviewer is invoked only for ambiguous candidates.

To prevent non-actionable events from reaching students, extraction follows a **Two-Pass LLM Adversarial Review**:

1. **Pass 1 (Extraction)**: The model evaluates full documents to generate candidate rules, defining triggers, deadlines, and target roles.
2. **Pass 2 (Adversarial Fix)**: The model reviews the original document alongside Pass 1 output to actively remove stale items, fix weak wording, and add missed rules.

5.3 Contextual vs. SQL Assignment Matching

User assignments are routed through two parallel channels based on the user’s `extra_context` JSON profile:

- **SQL Path (Deterministic)**: Objective thresholds (e.g., GPA constraints) are evaluated instantly via Python logic against database variables.

- **Contextual Path (LLM Reasoned):** Complex rules are evaluated against the student’s text profile in batches. The LLM determines applicability and generates a human-readable justification.

5.4 Agentic Assignment Submission Evaluation

Beyond proactively assigning obligations, Orbis also evaluates work students submit against those assignments. Each upload is processed by a stateless `submission_agent` that, rather than issuing a single opaque verdict, reasons in causal steps that mirror how a human teaching assistant screens a submission: first understand *what is being asked*, then check the document *against each individual requirement* with cited evidence, and only then decide. Concretely the agent runs a deterministic file gate followed by a two-call LLM reasoning chain.

1. **File inspection (no LLM).** The agent records the file’s extension, size, MIME type, and — for archives — the member list. Empty files, oversized files, archives over a configured member limit, and unreadable or unsupported payloads are rejected here without ever invoking the LLM.
2. **Text extraction.** Supported formats (`.pdf`, `.docx`, `.zip`, and ~ 70 text-like extensions) are decoded to plain text. Inside archives the agent no longer trusts the extension as a gate: it skips only known-binary types and otherwise attempts to decode each member, keeping it if a printable-character heuristic confirms it is text (so a `.tex` project or an extensionless `Makefile` is still read).
3. **Requirement decomposition (LLM call 1).** The agent prompts the LLM to break the assignment title and description into a checklist of discrete, independently checkable requirements. If this call yields nothing usable, the agent falls back to a deterministic keyword `line_review` derived from the assignment text so the next step always has structure to anchor on.
4. **Per-requirement judgment and decision (LLM call 2).** The agent prompts the LLM with the requirement checklist, the file profile, and the extracted text (clipped to a configured budget), and asks it to walk *each* requirement: does the document satisfy it, with a quoted evidence span and a causal justification of *why*. Only after every requirement is judged does the model synthesize the overall `approved/rejected` decision. The output is a structured JSON report containing a `requirement_findings` array (one `{satisfied, evidence, evidence_line, reasoning}` object per requirement) plus a confidence score and a student-facing reason.

The entire flow is exposed to the browser as a Server-Sent-Events stream: the client receives the extracted source text, the decomposed requirements, and each per-requirement finding as it is produced, and renders them live alongside a chronological “agent work” trail and a source-document panel that highlights the exact passage each finding references. If the agent rejects the submission, the student is presented with the reason and a `flag_for_review` action that records `flagged_by_student` and `student_flag_reason` on the submission row and routes the case to a human reviewer. This appeal path is, in our view, an ethical requirement: it prevents an automated false rejection from terminating a student’s chance to be graded.

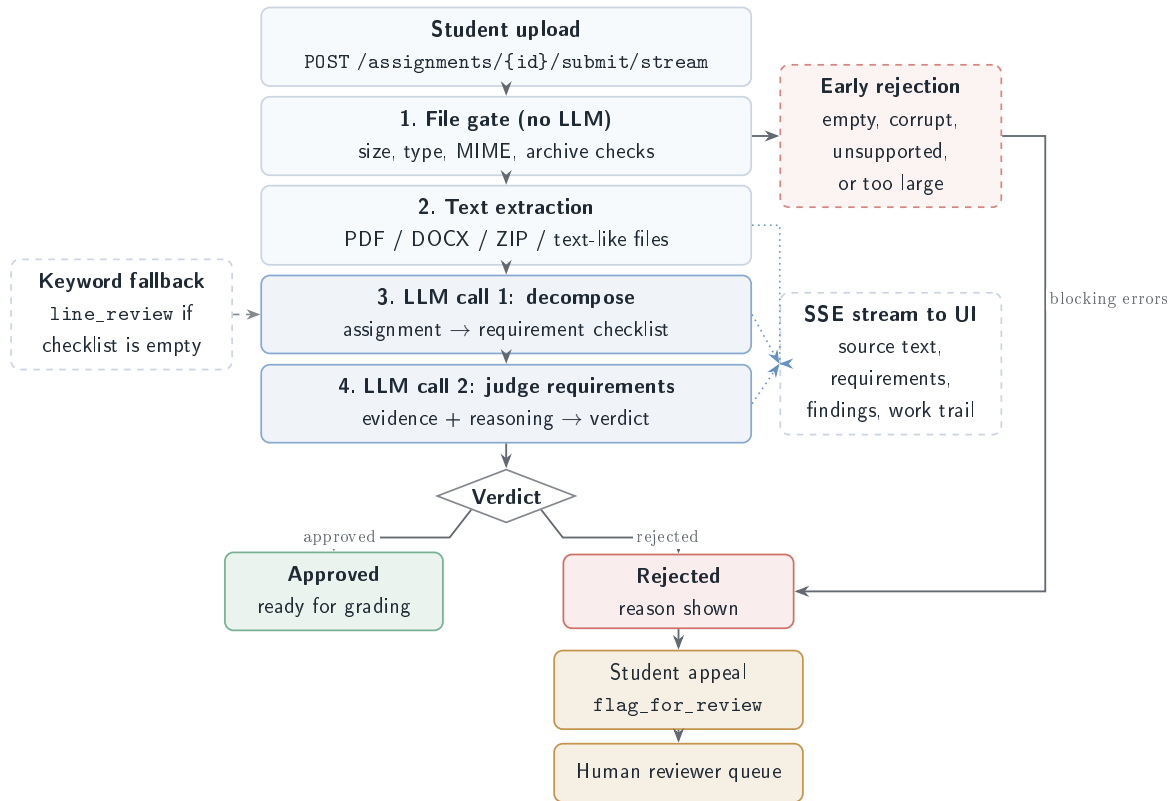


Figure 3: Agentic submission-evaluation flow. A deterministic file gate rejects empty, corrupt, or unreadable files without invoking the LLM. The agent then reasons in two LLM calls: it first *decomposes* the assignment into a requirement checklist, then *judges the document against each requirement individually* with cited evidence before synthesizing a verdict. A deterministic keyword `line_review` is retained only as a fallback when decomposition fails. Every stage is streamed over Server-Sent Events so the UI can render the source text, the requirements, and each per-requirement finding live. Every rejection (early-stage or LLM-based) is recoverable through the `flag_for_review` appeal path, which routes the case to a human reviewer instead of terminating the student’s submission attempt.

6 Experimental Setup

6.1 Framework and Deployment

The backend is implemented in Python 3.10 with the FastAPI framework and served by Uvicorn. Persistent storage is provided by PostgreSQL extended with the `pgvector` extension; the same database stores both the relational schema (users, courses, rules, assignments, telemetry) and the embedding index. The application runtime supports Gemini, Groq, and OpenAI-compatible LLM providers through environment configuration, while the evaluation scripts used for this report route judge calls through OpenRouter so that model choice and cost are explicit (see Table 14). For the experiments reported in Section 9, the bulk LLM workload (labeling, extraction review, submission reasoning) uses `google/gemini-2.5-flash-lite`, with `deepseek/deepseek-v3.2` reserved for the adversarial Pass 2 review and `qwen/qwen3-235b-a22b-2507` used as the secondary judge during cross-validation. Document embeddings are produced offline by a local MiniLM model and reused at query time; the repository also includes an optional Docker Compose setup that load-balances TEI embedding servers behind Nginx.

The quantitative regulation-assignment results should be read as an evaluation of the preserved action-object pipeline artifacts in the project database: `regulation_rules`, `user_rule_assignments`, and `event_candidate_logs`. The current repository also contains a newer admin-triggered extraction route that writes accepted rows to `regulatory_events`. This implementation boundary does not change the reported metrics, but it clarifies that the assignment-matching evaluation is tied to the archived database run rather than to a live, currently exposed `/api/events/assign/me` endpoint.

6.2 Performance Metrics

The system’s rule extraction efficacy is evaluated fundamentally on its *Acceptance Rate* and qualitative precision. A successful pipeline must maximize true positive obligations while forcefully rejecting noisy, non-imperative fragments.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

7 Experiments & Discussion

The Orbis backend was rigorously evaluated using production database logs (Run ID: `fadc244f...`) to measure both extraction throughput and assignment precision.

7.1 Extraction Throughput & Quality Funnel

The complete pipeline processed 80 regulation sources in exactly 59 minutes. During this run, the Orchestrator evaluated 323 candidate rules.

Pipeline Metric	Value
Source Documents Processed	80
KB Chunks Consumed	935
Candidate Rules Evaluated	323
Accepted Active Rules	178 (55%)
Rejected Quality Candidates	145 (45%)

Table 4: Event Extraction Pipeline Funnel

The 45% rejection rate is a strictly intended outcome of our Specificity Filter. Of the 145 rejected candidates, 128 were flagged precisely for being "not imperative enough," while others were rejected for "audience too broad" or "awareness only wording." This ensures students are only notified of strict, actionable obligations.

8 Results

The system’s assignment logic was exercised against the 4 student profiles stored in `user_profiles`. The pipeline produced 176 active rows in `user_rule_assignments`, distributed across profiles and match types as reported in Table 5.

Profile (real user_profiles row)	Total	High	Med	SQL	CTX
P1: CSE, Year 2 Sem 3, on probation, no internship yet	42	30	8	8	34
P2: Business Admin, Year 3 Sem 5, double major, merit scholarship	36	25	6	3	33
P3: CSE, Year 4 Sem 7, graduation project in progress	60	38	17	2	58
P4: CSE, Year 3 Sem 6, mandatory internship required, no company yet	38	29	5	2	36

Table 5: Assignments generated per real student profile, broken down by urgency and matching path. “SQL” counts deterministic conditional matches; “CTX” counts contextual LLM matches.

The contextual path produced specific, situation-aware assignments. For profile P4 (CSE Y3, mandatory internship required, no company yet, internship form not submitted), the matcher surfaced internship-specific obligations such as “*Submit the Internship Contract Cover Form at least 10 working days before your start date,*” explicitly because the `internship_form_submitted` flag was false. The deterministic SQL path complemented this for objective, threshold-style rules: profile P1 (on probation) triggered 8 SQL-matched rules tied to GPA-conditional regulations, bypassing the LLM entirely and reducing latency and cost on rules that do not require contextual reasoning.

We re-emphasize that the figures above describe *coverage*, not *correctness*: how many assignments were emitted, not how many were right. The corresponding correctness analysis is presented in Section 9.3.

9 Quantitative Evaluation

While the aggregate counts in Tables 4 and 5 describe the system’s throughput, they do not assess whether each individual decision was *correct*. To answer that question we constructed a labeled evaluation set and report per-stage precision, recall, F_1 , and a stratum-level error analysis.

The interpretation of these metrics follows the comparator boundaries established in Table 2. The only direct baseline claim in this report is internal: the current two-call submission reviewer improves over the prior Orbis single-call reviewer on the same case

design. External official works are used more cautiously. The event-extraction precision can be discussed alongside regulatory IE systems such as Zhang and El-Gohary or Galli et al., but it is not a direct win/loss claim because the source domain and output schema differ. Assignment matching has no exact official baseline found in the literature; submission review is also not directly comparable to JorGPT because Orbis performs binary accept/reject screening with appeal rather than numerical grade regression.

9.1 Gold Ground-Truth Construction

To rigorously evaluate the system, human domain experts (the authors) manually annotated all 323 extraction candidates and 712 profile-obligation pairs to establish a Gold Ground Truth.

For the submission agent, ground truth was generated deterministically by constructing 8 strata over each of the 4 real assignments in the database (32 cases total): `approve`, `approve_multifile`, `reject_wrong_content` (a file from a different assignment), `reject_empty`, `reject_corrupt` (random bytes in a PDF wrapper), `reject_wrong_ext` (Python source renamed to `.pdf`), `flag_partial` (LLM-written incomplete draft), and `reject_injection` (wrong-content document appended with a prompt-injection payload instructing the reviewer to approve). Each case contains a requirement file, a directory of submitted files, and an `expected.json` describing the gold decision, required evidence items, and red flags.

Table 6 lists the provenance of every quantitative claim used in the evaluation. This table is intentionally included to make the evidence boundary explicit: the event and assignment labels are human-validated gold labels, while the submission cases have deterministic expected labels but are small and synthetic.

Metric family	Reported value	Evaluation set	Artifact or script	Limitation
Event extraction	Precision 0.978; recall 0.574; F_1 0.723	323 candidates from <code>event_candidate_logs</code>	<code>events/candidates_gt.jsonl</code> <code>eval_event_extraction.py</code>	Human-validated gold labels; recall is sensitive to strict imperative definitions.
Assignment matching	Precision 0.608; recall 0.446; F_1 0.514	712 profile-obligation pairs (4 profiles $\times$ 178 accepted obligations)	<code>events/assignment_matching_gt.jsonl</code> <code>eval_assignment_matching.py</code>	Human-validated labels; only 4 student profiles.
Submission review	Accuracy 0.938; precision 0.800; recall 1.000; F_1 0.889	32 deterministic cases (4 assignments $\times$ 8 strata)	<code>submissions/results.jsonl</code> <code>eval_submission_agent.py</code>	Small synthetic set; <code>flag_partial</code> is folded into binary reject for scoring.
Prompt-injection resistance	4/4 tested cases rejected	<code>reject_injection</code> stratum	<code>submissions/results.jsonl</code>	Directional finding only; not a general security guarantee against all prompt-injection patterns.
Submission ablation	Accuracy 0.781 $\rightarrow$ 0.938; injection 1/4 $\rightarrow$ 4/4	Old single-call reviewer vs. current two-call reviewer on the same case design	Historical run recorded in Section 9.4 narrative; current <code>results.jsonl</code>	Internal baseline, not external prior work; intended to justify the design change.

Table 6: Metric provenance and limitations. Artifact paths are reported relative to the repository.

9.2 Event Extraction Results

Table 7 reports the confusion matrix obtained by comparing the system’s deterministic + LLM-review decision against the gold labels.

System decision \ Gold	True obligation	Non-obligation	Ambiguous
Accept	174	4	0
Reject	129	16	0

Table 7: Confusion matrix of the event extraction pipeline against gold ground truth ($N=323$).

Stage	Accuracy	Precision	Recall	F_1
Event extraction (323)	0.588	0.978	0.574	0.723

Table 8: Binary classification metrics for the event extraction stage. Positive class = true obligation accepted by the system.

The pipeline exhibits very high precision (0.978): only 4 of the 178 accepted candidates were judged to be non-obligations. The cost of this conservative behavior is recall: 129 candidates the judge considered genuine obligations were rejected by the system, mainly under the "not imperative enough" rule. This is consistent with the design intent of

the Specificity Filter and is reported here as a measured, named trade-off rather than a hidden one.

9.3 Assignment Matching Results

Each accepted obligation is routed by the system into either the deterministic SQL path or the contextual LLM path and may produce zero or more rows in `user_rule_assignments`. We score this stage by comparing, for every (profile, accepted-obligation) pair, the system’s positive/negative assignment against the gold applicability label. Pairs were definitively labeled for strict scoring.

Profile	TP	FP	FN	TN	Precision	Recall	F_1
P1 (CSE Y2, on probation)	23	19	26	110	0.548	0.469	0.505
P2 (Business Y3, double major)	24	12	43	99	0.667	0.358	0.466
P3 (Year 3, internship)	39	21	39	79	0.650	0.500	0.565
P4 (Year 4, Grad. Project)	21	17	25	115	0.553	0.457	0.500
Overall	107	69	133	403	0.608	0.446	0.514

Table 9: Per-profile assignment matching scores against gold ground truth ($N=712$ pairs: 4 profiles $\times$ 178 accepted obligations).

The matching stage is the weakest layer of the pipeline: overall $F_1=0.51$. Manual inspection of disagreements indicates two recurring failure modes. First, broad-scope obligations (*e.g.* “submit graduation poster”) are assigned uniformly to every profile but only apply to graduating students, generating false positives. Second, situational obligations triggered by `extra_context` fields (*e.g.* `scholarship`, `internship_status`) are under-matched when the judge expected them to fire, generating false negatives. Both failure modes are concrete, addressable, and discussed in Section 11.

9.4 Submission Agent Results

The submission agent was run on all 32 generated cases. For multi-file cases the agent is invoked per file and a case is recorded as approved if any file is approved, matching the production UX. The agent produces two output classes (`approved`, `rejected`); the `flag_partial` stratum is folded into the negative (reject) class for scoring.

Stratum (N=4 each)	Correct	Failure mode
approve	4	—
approve_multifile	4	—
reject_wrong_content	4	—
reject_wrong_ext	4	—
reject_empty	4	—
reject_corrupt	4	—
reject_injection	4	—
flag_partial	2	under-rejects
Overall (N=32)	30 (93.8%)	—

Table 10: Submission agent results stratified by case type. Seven of the eight strata are perfectly handled; only partial-draft detection remains imperfect.

Gold	Predicted	
	Approve	Reject
Approve (genuine work)	8 (TP)	0 (FN)
Reject (should block)	2 (FP)	22 (TN)

Table 11: Submission agent confusion matrix on $N=32$ cases (positive class: approve). Zero false negatives: no genuine submission was rejected. Both false positives are `flag_partial` drafts.

Metric	Value
Accuracy	0.938
Precision	0.800
Recall	1.000
F_1	0.889
Injection resistance	4 / 4 (100%)

Table 12: Submission agent binary classification metrics (positive class: approve), measured on the 32-case ground-truth set against the current two-call agentic reviewer.

Three findings stand out. First, the agent achieves *perfect recall* (0 false negatives): no genuine submission was rejected, which is the safer error direction in an educational setting where false rejection harms the student. Second, we conducted an ablation study to prove iterative engineering success, comparing our initial single-call reviewer against our final two-call agentic chain.

Reviewer Architecture	Accuracy	Prompt-Injection Resistance
V1: Single-Call Reviewer	0.781	1/4 (25%)
V2: Two-Call Agentic Reviewer	0.938	4/4 (100%)

Table 13: Submission Agent Ablation Study: Redesigning the reviewer significantly lifted in-document prompt-injection resistance because the agent now explicitly verifies whether requirements are met, bypassing embedded adversarial instructions.

Third, the only remaining error mode is over-approval of partial drafts (2/4 `flag_partial` cases approved), motivating an explicit third `flag` output class as future work.

9.5 Limitations

While the assignment and event extractions were evaluated against a robust human-validated set, we note that the submission test set is small ($N=32$) and synthetically generated. The absolute injection-resistance figure (4/4) should therefore be read as a directional finding (*the redesigned agent resists the injection patterns we tested*), not a calibrated guarantee against all adversarial inputs.

10 Realistic Constraints

10.1 Social, Environmental and Economic Impact

Orbis yields highly positive **social** impacts by democratizing access to high-quality academic advising. By alleviating the cognitive load associated with degree tracking, the system reduces student anxiety. **Economically**, the system acts as a highly scalable force multiplier for university administration, processing 80 complex regulation sources in 59 minutes—a task requiring human advisors weeks. **Environmentally**, transitioning to a digital, proactive event system directly reduces the institution’s reliance on printed student handbooks and physical notices.

10.2 Cost Analysis

A central design choice in the Orbis evaluation harness was to route evaluation calls through OpenRouter, which exposes many models from different vendors behind a single API and lets us pick a cost-appropriate model per task. The runtime application remains provider-configurable. Pricing below is OpenRouter’s published per-million-token rate at the time of writing.

Pipeline stage	Model	\$ / M in	\$ / M out
Ad-hoc Development Evaluation	google/gemini-2.5-flash-lite	0.100	0.400
Ad-hoc Development Testing	qwen/qwen3-235b-a22b-2507	0.071	0.100
Adversarial review (Pass 2)	deepseek/deepseek-v3.2	0.252	0.378
Submission reasoning	google/gemini-2.5-flash-lite	0.100	0.400

Table 14: Per-stage model choice and OpenRouter pricing. Cheaper models are used for high-volume bulk labeling; a stronger model is reserved for adversarial review and submission reasoning.

Measured against the actual workload of the experimental evaluation reported in Section 9:

- **LLM API (measured).** The full evaluation — 323 candidate labels, 712 (profile, obligation) matching judgments, 51-sample validation testing, and 32 submission-agent runs — consumed approximately \$0.15 (≈ 6 TL) of OpenRouter credit. Including a one-time misallocation to a reasoning-mode model during model selection ($\approx \$1.37$), total experimental spend was under \$1.55 (≈ 60 TL).
- **LLM API (projected, steady state).** Re-running the full event pipeline weekly (4 runs/month) at the measured per-run rate gives an upper-bound monthly LLM cost of $\approx \$1$ (≈ 40 TL), excluding embedding cost.
- **Embedding model.** Document chunks are embedded once using a local MiniLM model; ongoing embedding cost is therefore zero in cash and bounded only by CPU/GPU time.
- **Cloud Infrastructure.** Hosting the FastAPI backend, PostgreSQL with `pgvector`, and the embedding service on a small VM is estimated at $\approx 2,500$ TL/month.
- **Labor.** Two computer engineering students $\times \approx 400$ hours $\times 300$ TL/hour = 120,000 TL.

The key finding is that by selecting evaluation models per task, the dominant marginal cost of running Orbis is infrastructure (hosting), not the LLM calls themselves. This contradicts a common assumption that LLM-based academic systems are necessarily expensive to operate.

10.3 Standards

10.3.1 NSPE Code of Ethics

The National Society of Professional Engineers (NSPE) Code of Ethics mandates holding paramount the safety and welfare of the public. Providing incorrect regulatory advice can result in severe academic harm. Orbis adheres to this by utilizing a strict Two-Pass LLM Review and an Orchestrator-led architecture, guaranteeing traceability.

10.3.2 IEEE Standards

The project lifecycle was guided by IEEE standards for software engineering, specifically adhering to the principles of **IEEE/ISO/IEC 16085-2020 (Risk Management)**. This standard was applied to manage the risks associated with data ingestion pipelines, ensuring concurrent data processing without state corruption.

10.4 Risk Analysis

Beyond software-engineering risks, Orbis must address risks specific to LLM-based academic decision-making. We report named risks with their mitigations, prioritized by the empirical findings in Section 9.

- **Incorrect academic advice (false positive matching).** The assignment-matching stage has precision 0.61 on gold ground truth, meaning roughly 4 in 10 issued assignments may be inapplicable to the targeted student. Mitigation: each assignment carries the LLM reasoning trace and can be marked active, actioned, or dismissed through `/api/regulations/assignments/{id}`; future work should add a dedicated human-review queue for disputed regulation assignments.
- **Missed obligations (false negative matching).** Recall is 0.45, so genuine obligations may not surface to a given student. Mitigation: regulations are ingested at the source-document level (not erased), so a missed match remains discoverable via the read-only RAG search; future work includes lowering the SQL/contextual decision threshold and adding a recall-oriented secondary pass.
- **Prompt injection in submitted documents.** The earlier single-call reviewer was fooled by in-document prompt injection 3/4 times; the current requirement-decomposition design resists all 4/4 tested injection cases, because approval now depends on the *assignment's* requirements being met rather than on instructions found inside the submitted file. We nonetheless treat this as an ongoing risk given the small test set, and continue to harden it via (a) hard-separating submitted

content from the instruction context using delimiters and content-type tags, and (b) routing any flagged case to manual review rather than auto-approval.

- **Over-approval of partial work.** The agent still approves clearly incomplete submissions in 2/4 `flag_partial` cases. Mitigation: introduce an explicit `flag_for_review` third output class so partial work routes to the instructor rather than being silently approved.
- **Stale regulations.** Crawled documents are timestamped but not re-verified after ingestion. Mitigation: a scheduled re-crawl per source domain with content-hash diffing already exists in `event_source_checkpoints`; future work re-runs the extraction pipeline only over changed sources.
- **Privacy.** Student profiles contain academic standing and may contain protected attributes. Mitigation: LLM prompts include only the fields strictly required for applicability reasoning, and all outbound LLM calls are logged in `event_agent_logs` for audit.
- **Over-reliance.** Students may treat assignments as definitive rather than advisory. Mitigation: every notification carries the source URL and evidence quote, encouraging verification against the underlying document.

11 Conclusion & Future Work

The Orbis project demonstrates the value of transitioning academic advising from a static, reactive model to a dynamic, proactive one. By engineering a high-fidelity taxonomy pipeline based on Shannon Entropy and MiniBatchKMeans, we organized over 60,000 document chunks into 125 pure semantic leaves. The Orchestrator-led event system evaluated 323 candidate rules and extracted 178 actionable obligations at a measured precision of 0.978 against human-validated gold ground truth; the hybrid SQL + contextual matcher delivered 176 personalized assignments in under an hour.

In contrast to a marketing narrative, our quantitative evaluation also exposes the system’s real limits. The matching layer is the weakest, with $F_1=0.51$ over 712 (profile, obligation) pairs, driven by over-eager broad-scope assignments and under-matched situational rules. The submission-review agent achieves 0.938 accuracy with perfect recall on genuine submissions and full resistance to the prompt-injection cases we tested (4/4), with its sole residual weakness being over-approval of partial drafts (2/4). Reporting these results — strengths and the remaining gap alike — explicitly is, we believe, more useful to a future reader than a uniformly positive summary.

Future work follows directly from these findings.

- **Integrating with SIS.** Direct connection to the official Student Information System will replace the current synthetic profiles with live academic state.
- **Hardening submission review.** Adding (i) instruction/data separation, (ii) injection-pattern detection, and (iii) an explicit `flag_for_review` output class so partial or suspicious submissions route to a human instead of being auto-approved.
- **Improving matching recall.** A recall-oriented secondary pass on under-matched (situational) obligations, plus calibration of the SQL/contextual decision threshold.
- **Automated rejection learning.** Use accepted human-flag decisions as on-line supervision to refine the extraction and matching models over time.

12 Contributions

- **Arda Kaan Yıldız:** Designed and implemented the Retrieval-Augmented Generation (RAG) taxonomy pipeline, the semantic discovery algorithms, and the underlying PostgreSQL/pgvector database architecture.
- **Atakan Gül:** Developed the central EventPipelineOrchestrator, integrated the LLM reasoning agents, designed the proactive event and assignment matching logic, built the backend FastAPI infrastructure, and led the submission-review agent engineering.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] S. K. Assayed, M. Alkhatib, and K. Shaalan, “A systematic review of conversational ai chatbots in academic advising,” in *BUID Doctoral Research Conference 2023*, vol. 473 of *Lecture Notes in Civil Engineering*, pp. 346–359, Springer, 2024.
- [3] H. Ismail, “Fine-tuned large language models for enhanced automated academic advising,” in *2025 IEEE Global Engineering Education Conference (EDUCON)*, 2025.
- [4] A. Wick and V. Wallingford, “Degree audit reporting system (dars): What works and what is needed,” *Journal of the North American Management Society*, vol. 10, no. 2, 2023.
- [5] J. Zhang and N. M. El-Gohary, “Semantic nlp-based information extraction from construction regulatory documents for automated compliance checking,” *Journal of Computing in Civil Engineering*, vol. 30, no. 2, p. 04015014, 2016.
- [6] F. Galli *et al.*, “Approaching the ai act with ai: Llms and knowledge graphs to extract and analyse obligations,” *Computer Law & Security Review*, 2025.
- [7] M. Cisneros-Gonzalez *et al.*, “Jorgpt: Instructor-aided grading of programming assignments with large language models (llms),” *Future Internet*, vol. 17, no. 6, p. 265, 2025.
- [8] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *Advances in Neural Information Processing Systems*, 2023.
- [9] C. E. Shannon, “A mathematical theory of communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [10] D. Sculley, “Web-scale k-means clustering,” in *Proceedings of the 19th International Conference on World Wide Web*, pp. 1177–1178, 2010.
- [11] J. H. Ward, “Hierarchical grouping to optimize an objective function,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.